

SmartCare v0.1 - Initial Engineering Brief and AI Activity Card

A2 Case Study Stage 1 student resource

SmartCare scenario

SmartCare Community Clinic currently uses spreadsheets and paper records to manage patients and appointments. The client says: 'We need software to help manage patients, practitioners and appointments.' This is not yet a complete specification.

Initial Engineering Brief

1. Problem summary

Write approximately 100 words.

SmartCare currently has several operational problems due to workplace practices where records are made both digitally and manually, causing issues including but not limited to: inconsistencies in patient information, duplicate appointment bookings, difficulties locating patient records, and so on. As a result, the client primarily requests a simple software system that can improve patient, practitioner and appointment management. In addition, it is known that the current requests from the client are broad, hence the exact system requirements have not been completely specified. To help solve the issue, the software engineer first needs to identify stakeholder needs and clarify missing requirements. The software engineer also needs to ask questions regarding the identified issues to further clarify the client's requirements.

2. Initial stakeholders

Stakeholder	Possible need
Patients	Have accurate patient and appointment information
Healthcare Practitioners	View appointment information and details
Receptionists	Manage patient information and appointments efficiently
System administrator	Manage and maintain system functionality

3. Initial features

Feature	Confirmed or provisional?	Why?
Appointment management	Confirmed	Core feature of the client's program requirements to solve the current issue
Patient management	Confirmed	Client requested software to support patient management
Practitioner management	Confirmed	Client requested software to support practitioner management
Patient search	Provisional	Not requested by client, but can help address issue in regards to finding patient information
Appointment cancellations	Provisional	Not requested by the client, but is an existing problem in their clinic.

4. Questions for the client

1. As a result of managing bookings, how will cancellations be addressed in the system?
2. Who should be able to manage appointment bookings (edit, modify, etc.)
3. What information needs to be saved when closing the program?

5. What we do not yet know

1. We do not yet know whether a practitioner can have more than one booking at the same time
2. We do not yet know who is allowed to manage appointment bookings.
3. We do not yet know whether appointment cancellations are to remain in the system

AI Activity Card - Ask, Check, Explain

Before AI

What do I think the code does? What problems can I already identify?

What it does	Problems identified
Adds appointments into a list	Does not validate practitioner_name or appointment_time
Utilises dictionaries that contains the patient's name, the practitioner's name and the appointment time	Code does not save inputted appointment information, which ultimately causes new inputs to disappear when closing the application. Appointment_time does not imply that it includes the date
Uses a book_appointment() function to substitute in the dictionary	
display_appointments() lists saved appointments (via book_appointment())	

AI request

Act as a tutor. Explain this code and identify potential problems. Do not provide a complete replacement. Ask me questions that help me reason about the solution.

Evaluate

Suggestion	Useful	Unclear	Incorrect	Out of scope
Validate all fields	x			
Prevent double-booking	x	x		
Prevent duplicate entries	x	x		
Add appointment cancellation		x		
Add appointment updates				x
Search functionality	x			x

Decide

For each significant suggestion: Accept / Modify / Reject / Keep unverified.

Suggestion	Accept / Modify / Reject / Keep unverified	Why?
Validate all fields	Accept	Prevents empty fields from being added into the list
Prevent duplicate entries	Keep unverified	Prevents data redundancy, currently unsure how to implement however as it may require a complicated process
Prevent double-booking	Keep unverified	No clarification about the clinic's scheduling rules and whether a practitioner can have more than one booking at one
Add appointment cancellation	Keep unverified	AI suggests this function by removing the appointment from the list, but currently unsure whether cancellations are to remain in the system or determined with a status
Add appointment updates	Reject	Complicates the system
Search functionality	Reject	Could be helpful, however may be difficult to implement and therefore complicate the system.

Verify

Testing normal input:

- Filled out details regarding patient_name, practitioner_name and appointment_time and the code runs without any errors.
- Ran the code without any inputs (no book_appointment() to run) in which the code printed out *"No appointments recorded."*

Testing unusual input:

- book_appointment('Bob Johnson') → TypeError
- book_appointment('Bob Johnson', "", "") → Patient: Bob Johnson | Practitioner: | Time:
- book_appointment("", "", "") → ValueError: Patient name cannot be empty

Comparing requirements:

Previous activities identify stakeholder requirements and compare them to AI suggestions. As a result, any AI suggestions that were out of scope are rejected whilst the relevant requirements are to be considered when implementing into the code.

Explain

Can I explain the final code without reading the AI response? What do I still need to understand?

The final code utilises a list to store appointments in which the function *book_appointment()* substitutes values such as the *patient_name*, *practitioner_name* and *appointment_time*. In addition, *book_appointment()* also validates the *patient_name* to ensure the field is not empty. The code then uses the function *display_appointments()* to output statements that list all the appointments in the *appointment* list.

I still need to understand basic Python fundamentals and relevant keywords in order to be able to implement verified requirements into the system to ultimately satisfy user needs.